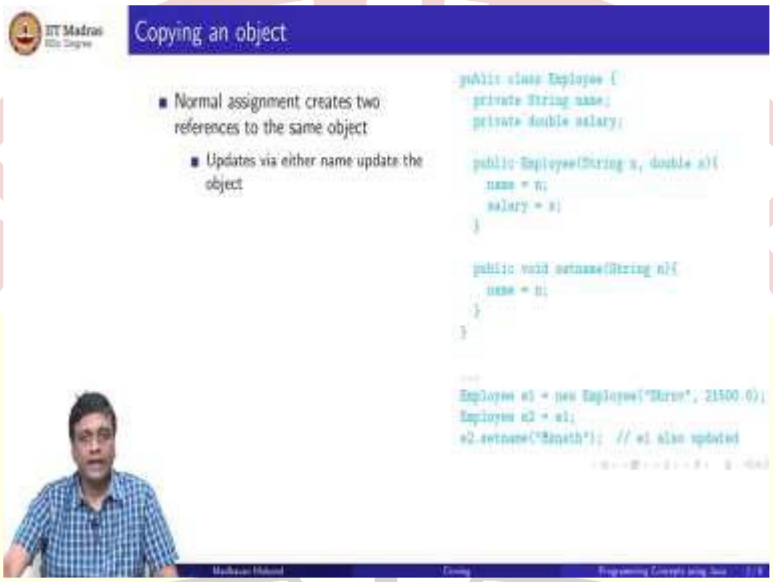
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Week 8
Cloning

(Refer Slide Time: 0.15)

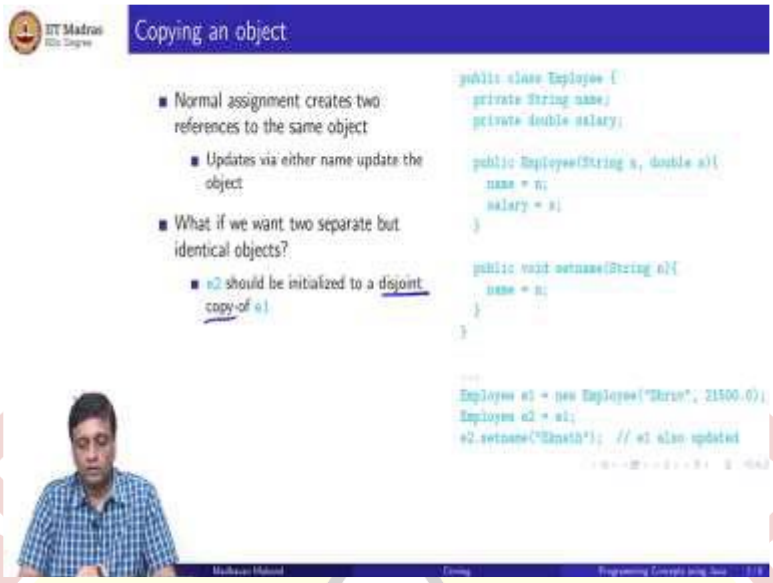


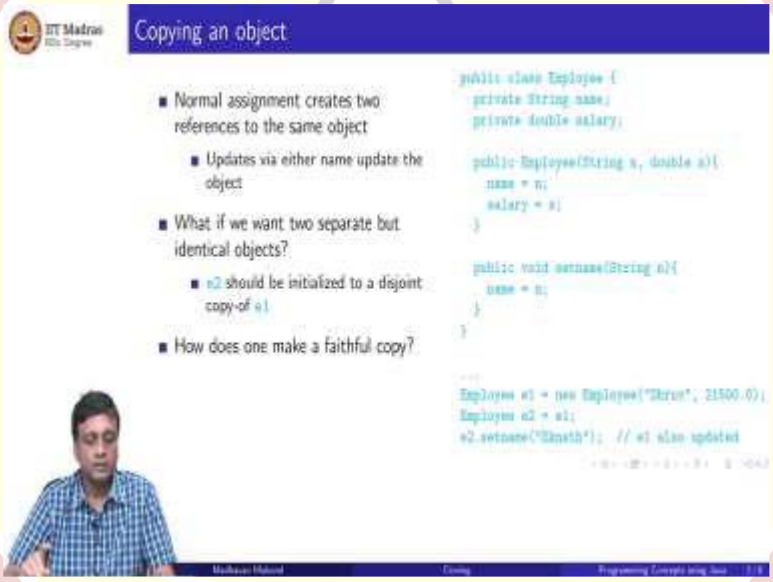
So, let us look at the problem of creating a copy of an object. So, normally, when we set up an object like so here, we have an employee class, so the employee has some instance variables, name and salary. And here is the constructor for this class. And we are able to, for example, in this case, update the name. So, we have an accessor, a mutator method, which rather allows us to update the name of such an object.

So, now if we set up an employee e1 with a particular name, and then we assign e2 to be the same as e1. And what happens is that we have only one employee object, so we have a name, and we have a salary. And we have e1 pointing to this and after this assignment, e2 is pointing to the same object. So, we have only one actual object in memory with 2 variable names pointing to the same thing.

So, now if I say, set the name with respect to e2, unfortunately, the name with respect to e1 also changes. So, there is a kind of a side effect, because this assignment operation only aliases these 2 names to point to the same object.

(Refer Slide Time: 1:26)

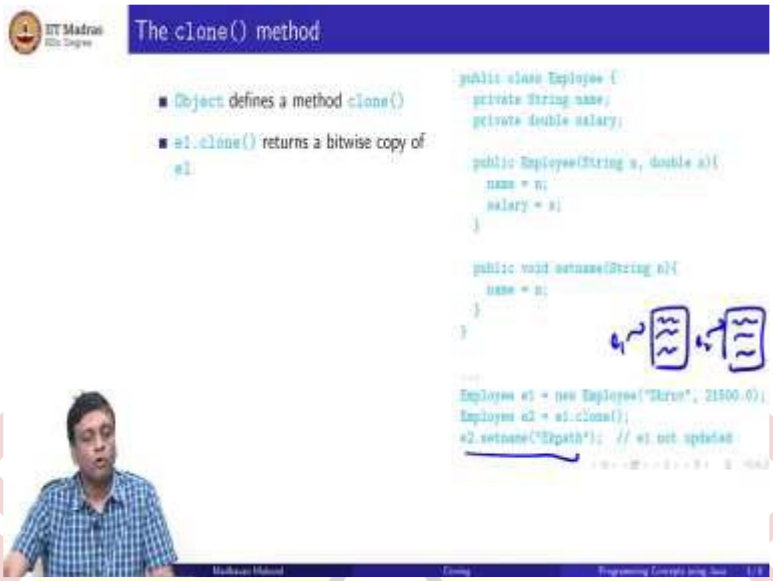




So, what happens if we really want 2 distinct copies, so we want e2 to be a faithful copy of e1? So, it takes all the values that are there in e1, and this is what happens when we have these simple variables like integers. If we say x = 5, and then we say y = x, then y also gets the value 5, but y and x are not pointing to the same location.

So, afterwards, if we manipulate y, x does not change. And if manipulate x, y does not change, so it is a one-time copy. So, we really want a copy of an object. So, we really want a disjoint copy more or less. So, in this case, what we can do is we can kind of clone an object.

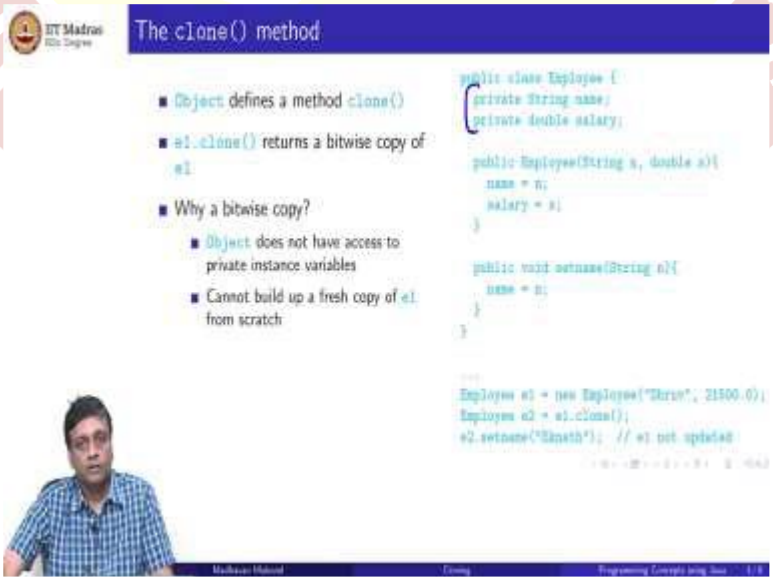
(Refer Slide Time: 2:08)



So, the object, capital object, the root of the Java hierarchy, defines a method called clone. And if you apply clone, then what happens is that you get the effect that we desired, which is, so the most obvious way to do this is if e1 is pointing to something, then we just copy whatever is there into a fresh piece of memory with the same size and make e2 point to that insect, so we return what is called a bitwise copy. So, bit by bit, we copy e1 into something new, and then we make e2 point to it.

So now, e2 is now referring to a physically different location in memory. So, e2 sets the name of its copy, the copy referred to by e1 is independent and does not change. So, this is really what we want to achieve. So, this is the clone method.

(Refer Slide Time: 2:58)

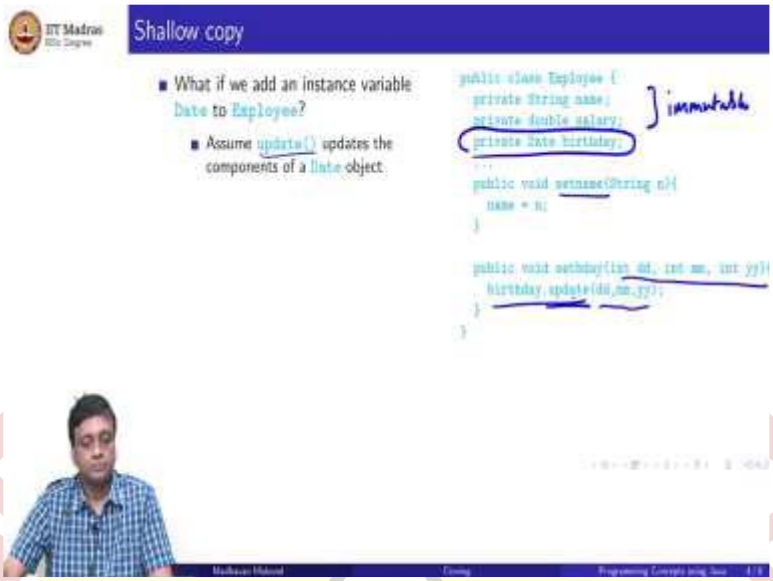


So, why do we want to create a bitwise copy? Well, from the perspective of the object hierarchy, object is sitting high up, and it does not know the private structure of any objects below it. In particular, it does not know that an employee objects data is organized in this way, nor does it know and what order it is or how it is stored. So, really, there is no meaningful way. So, the correct way to do it, perhaps is to construct a new object of type employee and populate it with the same instance variables as the original one.

Because after all, the state of an object is determined by the values of the instance variable. So, if you can create a new object, which has the same instance variables, and you copy the state one by one, then you will get a new copy of that object. But we cannot do it in that kind of elaborate way. Because we does not really know how these things are manipulated. So, we cannot necessarily extract all parts of the state and reconstruct constitute.

So, this bitwise copy is the only way that object as a clone method provided in general can work. It cannot create an object from scratch, so to speak, it has to just take it and make a faithful copy of it. So, this seems harmless enough. So, why is this not just a simple thing that we accept and move along with? So, why is the why do we need to discuss this clone method at all beyond this bitwise copy?

(Refer Slide Time: 4:21)

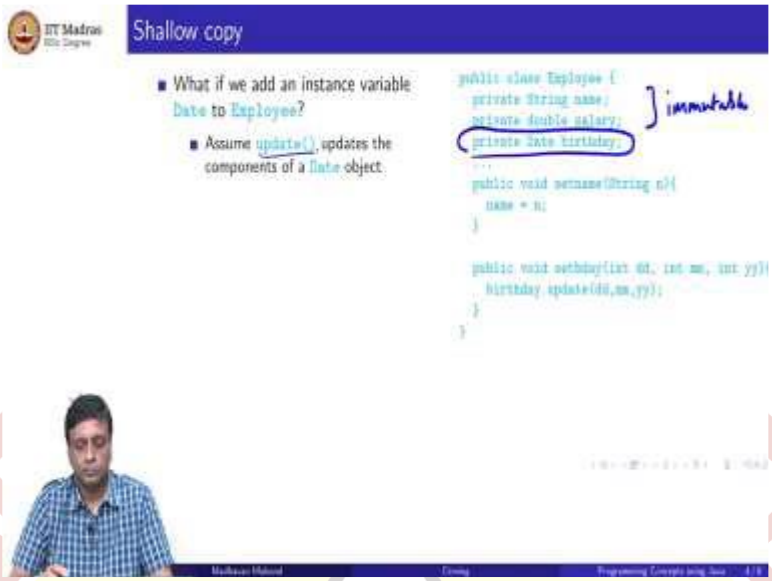


So, let us suppose this employee class is a slightly more elaborate class. And now in addition to the name and the salary, it also includes an instance variable, which is of type date. So, date now is slightly different from string and double because string and double these are both what we could call immutable values. So, if you cannot really update them without destroying them, but date is itself now another object.

So, suppose we have a function update which allows us to take a date, so a date presumably somehow records inside it, the day, month and year. So, suppose upgrade allows us to take an existing date and just change the values of day, month and year to its arguments. So, we have a, an update function which is available inside date, which takes 3 arguments, and internally changes, so it does not change the object, it changes the state of the object.

So, this is now in the employee class, just like we can set the name, we can set the birthday, by providing the set birthday command inside the employee class with 3 arguments which in turn, calls the date class birthday update with these 3 arguments.

(Refer Slide Time: 5:37)

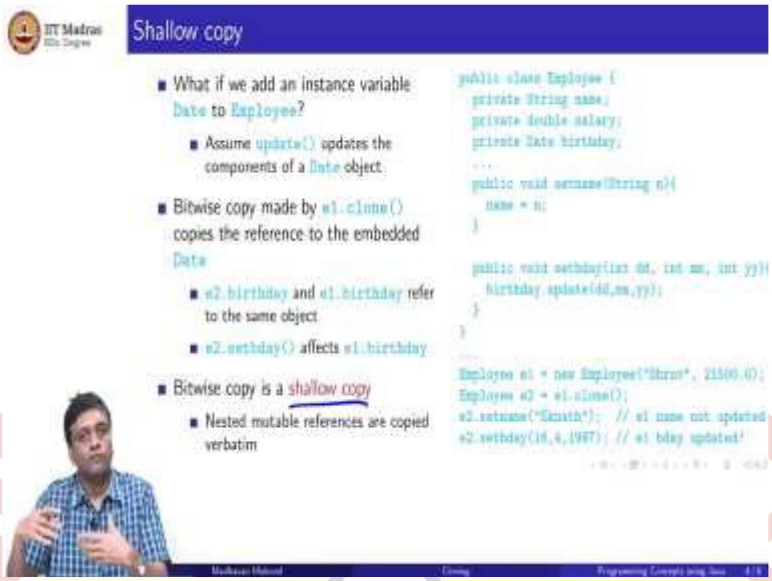


So now, if we make a bitwise copy, then in this bitwise copy, so remember that we had the name, we have the salary, and then we have the birthday. And here, we will now make a copy. So, this is e1, this is e2. And here we again have a name, a salary and a birthday. The problem is that d's are actually not here. So, this is the birthday of e1. And now the bitwise copy points to the same copy of the birthday as d1.

So, if an e2, I set the name, there is no problem because the name is distinct because I made a bitwise copy. But if I go through e2 and set the birthday is going to set the same birthday. So, therefore the e1 birthday is also going to change. So, the real problem with now bitwise copy is that it cannot faithfully make sure that the embedded objects are also copied bitwise.

Because we does not really know what is, remember the whole problem is that object does not know the structure of this representation of this employee object. So, it does not know which parts of it are the name, which parts are the salary, which part is a reference to the birthday. So, it just faithfully copies all the bits. So, it copies in the process the correct value of the name and the salary. But it also copies in reference that is a memory location in some sense of the birthday object that represents a birthday and the same memory location is now shared between the two.

(Refer Slide Time: 6:55)

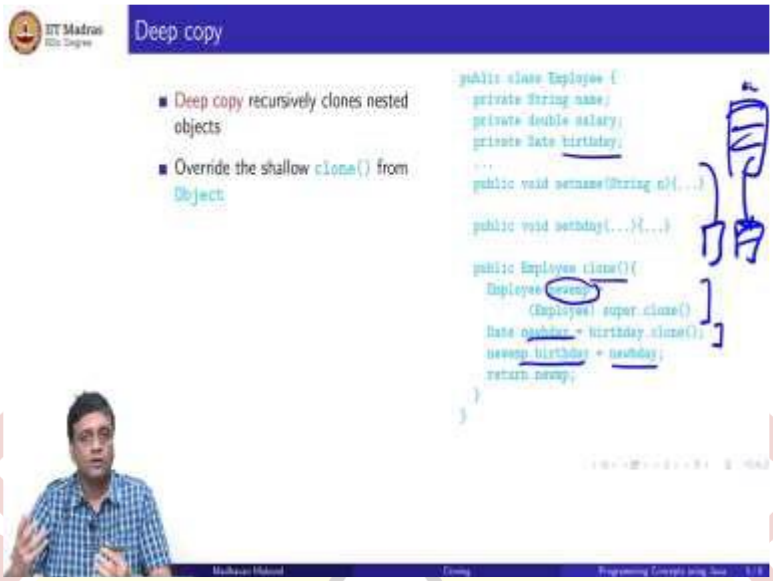


So, this is now referred to normally in the programming language literature as a shallow copy. So, we are copying at the top level, we take an object, and we make a faithful copy of all the bits in the object, but we have no semantic information to make a more detailed copy internally of the nested objects.

(Refer Slide Time: 7:18)

So, if we have a shallow copy, then obviously the corresponding improvement that we would like, is a deep copy. So, what does a deep copy do? It will look inside an object and see if there are nested objects. If there are nested objects, it will recursively make copies of those nested objects. So, in this case, the birthday will also be copied at the date level. So, there will be 2 objects of type date created, one for the original e1, and one for e2. So, it is a recursive copy, which goes down the semantic structure of the object.

(Refer Slide Time: 7:49)



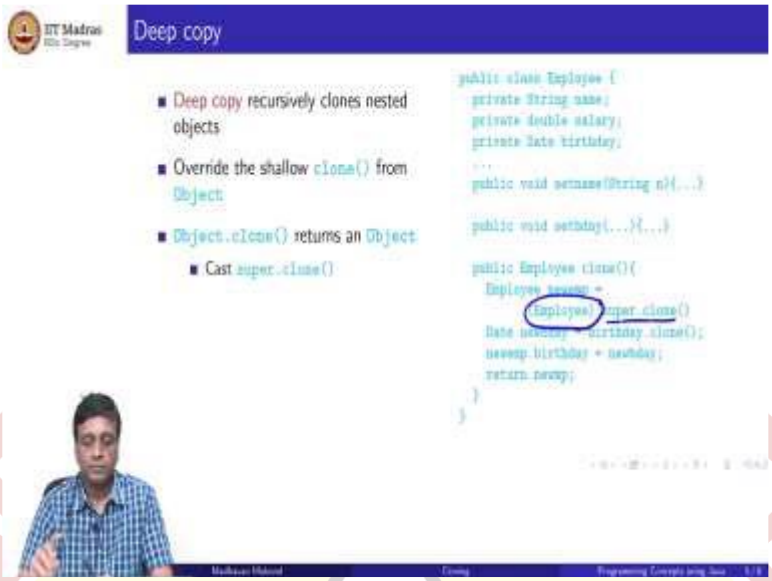
So, in order to get a deep copy, we have to implement it ourselves because as we said, object has no access to the internal structure of an arbitrary class. So, if we want employees to be capable of doing a deep copy, then we have to re-implement this clone. So, we have to override the clone method from object. So, we write a new clone. So, what does this clone method do?

Well, here is a simple way to do it, you first use the superclass clone. So, you take the object clone and make a bitwise copy. And now, you recognize that this birthday is also something that needs to be cloned. So, you make a bitwise copy of the birthday object. And now we assign, so we have created a new employee object, which is a bitwise copy of the old employee object.

Now, this includes a reference to the old birthday object, which we does not want. So, we make a new birthday object by keeping a bitwise copy of the birthday thing. And we assign the birthday value of the new employee to be this new birthday object to be created. So, we have basically done a nested copy. So, in first copy this whole thing. And then this thing was pointing here. So, we made a copy here and then we make this new e2 point here.

So, in this process, now everything is done, we does not have any overlap between e1 and e2, they both have a separate copy of the name, a separate copy of the salary. And there are 2 birthday objects, one pointed to by e1 and one pointed to by e2.

(Refer Slide Time: 9:18)



So, there are a few things to note in this. One is that the object version of clone returns an object because it has to be applied in many different contexts. So, it is the classical way of using the most generic return type object as the kind of polymorphic return type. So, what we want here is to make sure that when we clone an employee object, we get back an employee object.

(Refer Slide Time: 10:04)

And the other thing that we have done is that we have actually made this version of clone returns something which is different from object. So, this is normally not the case. So, we in fact, we said very specifically in one of the earlier lecture about the equals, and if we want to override the equals an object, we have to make sure that we write an equals method which returns a date and returns an object.

If we do it for date or employee, it will not override in the strict sense, but clone is a very special method. So, in the case of clone, when you override, if you change the return value, it is allowed. So, Java allows this in this particular case for this particular function. So, you have to do these 2 things, in order to do a deep copy, you have to first invoke the shallow copy of object and you have to invoke it repeatedly.

So, you have to make sure that you get these things. So, technically, possibly, this also should be a date, here which I missed, there should be one more cast here, because we even when we deep shallow copy the birthday part, again, it will return as an object. So, if we want to assign it to a date component here, then we have to make a suitable cast of that.

So, we are allowed to change the return type, and we must in some sense, cast it in order to get the return type to be the one that we want.

(Refer Slide Time: 11:18)

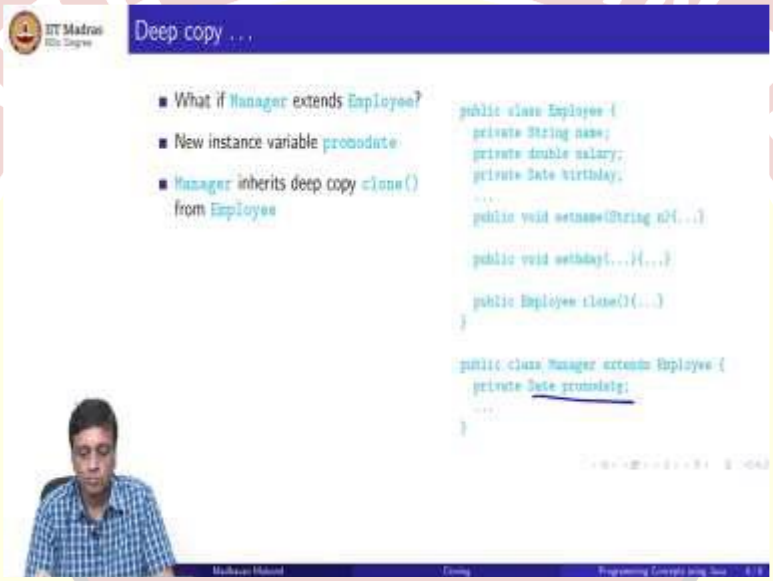
So, again, it would seem that deep copy is a very simple thing, of course, we have to leave it to the class itself and not to object because object cannot look into it. So, what could go wrong with a deep copy? So, the problem with this is the hierarchical nature of the Java class hierarchy. So, once I say that employee allows a deep copy, it has its own version of clone, then that clone is now visible below the hierarchy. So, if I now extend employee with a manager, then manager will also inherit the clone from the most recent ancestor, which is employee.

So, suppose now that we have this Manager class, which extends employee and it has not 1 date, but 2 dates, it gets a date from the parent class employee. So, it has these 3 private

fields, which it cannot even access. So, it gets this, but it also has a new date, say the promotion date. So, the manager has a birthday and also the date on which they got their promotion.

So, now, the problem is that if I invoke an employee copy on this, it will do a shallow copy of the whole object, then it will correctly assign a shallow copy of date in the new object. So, it has a new birthday, but it has no knowledge that there are sub classes like manager which have other objects below it, which need also to be deep copy.

(Refer Slide Time: 12:46)



So, the manager will inherit the clone, clone from employee, and there is no obligation for manager to do this again. So, if manager, the class manager inherits this clone, and it does not overwrite clone with a better deep copy clone, which uses the object, the clone of employee,

and then in addition, also deep copies this promo date, then we will have something where we end up with a shallow copy of the promotion date.

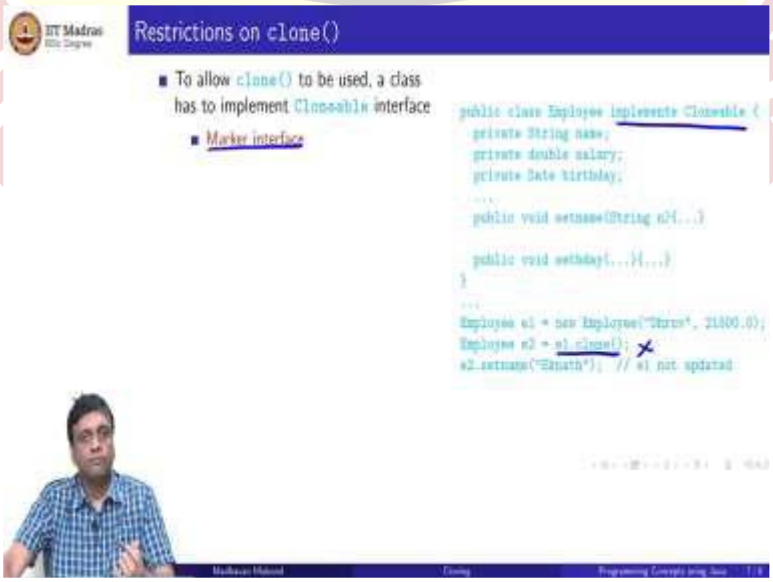
So, therefore, if I have 2 managers, now when I make a clock copy, if I 1 manager and I make a copy of that manager, then the second manager will have an independent salary name and independent salary and independent birthday, but we will share the joining date with the original one. So, if I update the joining date of one manager, it will update here. So, we are back to the same problem shallow copy.

And the problem here is there is no way for employee to know that this is wrong. And there is no obligation for manager to fix this because manager does inherit this clone, which is from employee and therefore, it does a partial deep copy. So, all these examples suggest that cloning is not a very straightforward operation.

So, shallow copy on its own has this aliasing problem. But deep copy is not necessarily a solution to the problem. Because if the sub classes do not understand that deep copy is working in a particular way and do not re-implemented, then you could have a deep copy, which leaves some things intact, but allows other things to be aliased.

So, as a result of these complications with cloning, what Java does is it makes it difficult to clone. In some sense, it puts some restrictions on the use of clone, so that a programmer who is using clone is aware that they are embarking on something which is potentially difficult to implement and could have unexpected consequences.

(Refer Slide Time: 14:32)

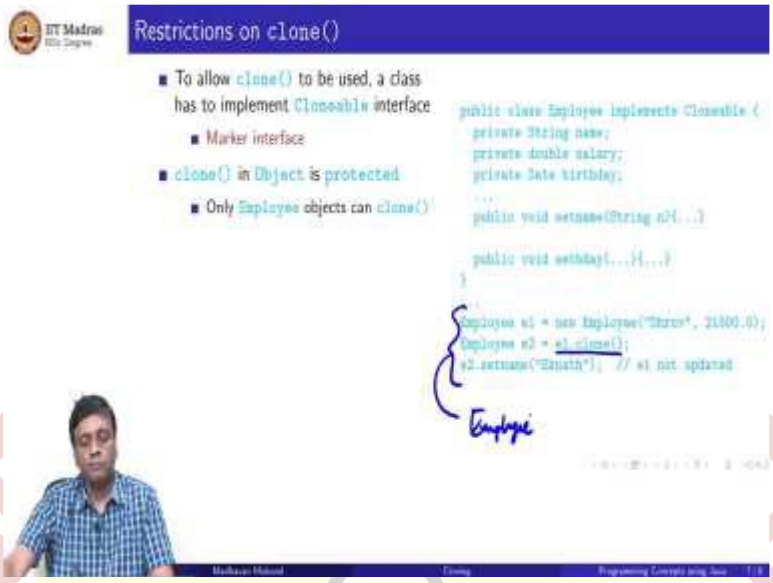


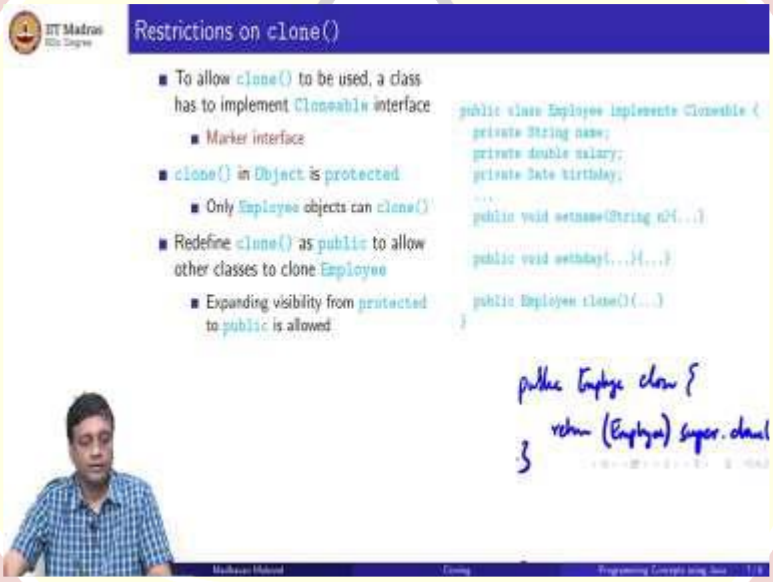
So, what are these restrictions? So, the first restriction is that a class that uses clone, which allows clone to be done on its objects must indicate this by implementing this interface called Cloneable. So, Cloneable, so we have to change the heading of our employee to allow Cloneable and if we allow Cloneable then this object clone can be invoked on it. If we does not invoke Cloneable we does not implement Cloneable, then this will give a compiler error saying that this class is not a candidate for clone to be applied.

So, Cloneable does not do anything, so Cloneable is just an indicator it is like a Boolean value, it is just saying that this particular class is an instance of the interface Cloneable. So, this we have seen before, but here, it is just a marker was called a marker interface, it does not have any functions that need to be implemented. So, to implement Cloneable this class has to do nothing else except declare that it implements clone. So, that is the first step.

So, with this first step, what you get to do is to use the object clone. Now, as we said, the object clone will be a shallow copy. So ideally, you would also like to use your own clone. So, we want to redefine, we want to overwrite the clone that said an object.

(Refer Slide Time: 15:51)





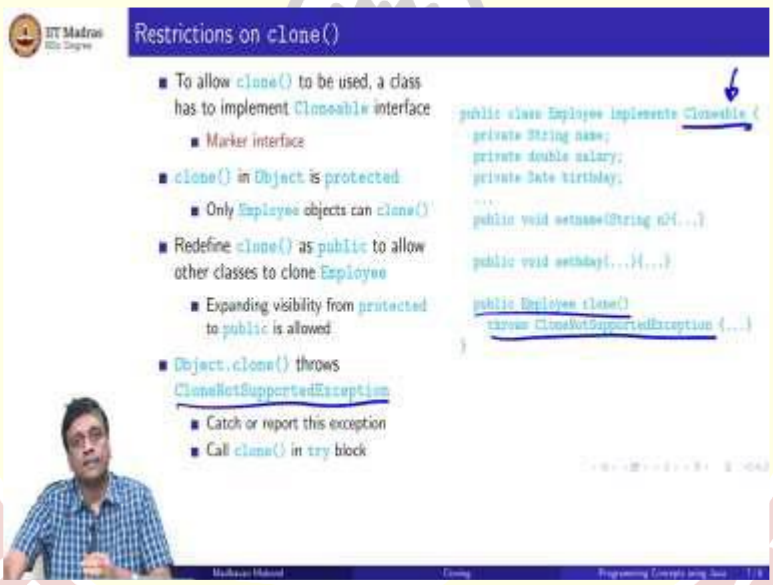
So, in order to make sure that this is done in a sensible way, object actually defines clone to be protected. So, remember, that protected normally says that it is visible only within the hierarchy. So, only something which is belongs to that class or is in the inherits from that class by extending it can invoke that thing. So, it is private within the tree rooted at a class.

But in this case, it is a little stronger than that again clone kind of has some peculiar specific restrictions associated with these common things. So, we saw that clone when you override you can expand its you can change its return type will here one thing that clone guarantees is that if you do not change its type from protected, then this will be allowed. Because an employee is applying clone to itself.

But you cannot invoke clone from outside. So, you have to actually, so, even this technically, this particular code has to be sitting inside employee. So, a class other than employee is not allowed to invoke the clone object if you leave it as protected. So, if you want other classes to clone not only must to implement Cloneable you must also redefine clone to be public. So, at a simple level, you can just redefine it to be public and make the effect of clone just super clone.

So, you can just say public whatever clone say employee say and you can say return a cast version of the object clone. So, this really does not change the meaning of clone it is still the shallow copy, but you have made it public. So, other classes can invoke clone on objects of type employee and these other classes will get back not an object, but an employee. So, expanding the visibility is allowed.

(Refer Slide Time: 18:00)



And the last thing is that remember that we are checking this Cloneable. So, in effect, the object clone is checking that the class that is calling clone, on which cloud clone is being called rather implements the Cloneable interface, and therefore there is a clone, not supported exception, which it could throw. So, if you actually call clone, and the class does not implement this interface, then you will get this exception.

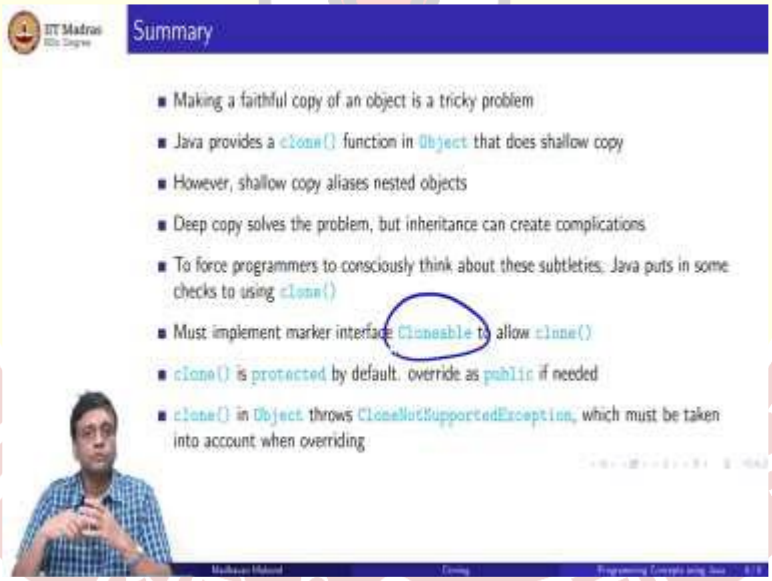
Now remember that when you extend or override a function with some kind of that throws an exception, a checked exception like this, then you must either continue to throw the checked exception, or you must fix it, you must catch it. So, therefore, if now we also write this clone thing, we must, the simplest way is to just continue to throw clone support exception.

So, we continue the same exception generating mechanism, which is there in object. So, if somebody calls this, they will also likely get this exception in case for some reason it is not available, of course, we have made sure that we do is only when we have done Cloneable. So, it is not likely to happen. But syntactically we cannot avoid this.

The other thing is that if I actually call clone from anywhere now, this means that any copy of clone, which I write anywhere, in principle throws this checked exception. So, we always have to use clone in a try block. So, this is a little bit of a nuisance, but again, it forces people who are using clone their code to think about what they are doing the code.

So, the whole purpose of all this is really to make it sort of difficult to use clone, under the assumption that clone is a tricky thing, and maybe you does not need to use it. So, it is debatable whether you actually need to clone objects like this or not. And so Java says, if you really insist on using the default clone mechanism, then make sure that you do all these extra things to understand that you are doing something which is potentially harm

(Refer Slide Time: 20:01)



So, to summarize, making a faithful copy of an object is not a simple thing. We have the shallow copy and deep copy, but making a faithful copy involves a lot of other semantic issues. So, Java provides the shallow copy through a function called clone an object. And if you do a shallow copy, of course, the problem with that is that if there are nested objects, then you will end up having an alias at that level.

So, even though the top level you will disambiguate the instance variables, you will get 2 copies of every instance variable. If the instance variable is pointing to an object, then you

will only get two copies of the address to the object. So, then you will have to make sure that you clone that instead and that is a deep copy. So, then this cannot be done at the level of object because object does not have access to the internal structure.

So, deep copy has to be implemented by the class. But now, if we implement deep copy at the class level, we saw like if employee has deep copy, and manager extends employee and manager adds more of these nested object type instance variables, and the employee core deep copy will not work for those. So, therefore, inheritance can still create problems with deep copy. And so to force programmers to consciously think about the subtleties, Java has put in these checks about using clone.

So, one of them is that it must explicitly implement this marker interface Cloneable. So, if you want to allow objects of a particular class to be cloned by anybody, including itself, you must implement this marker interface. So, this marker interface just says that you are aware, it is like signing a disclaimer, I am aware that this clone can be applied on this. It does not have any other obligations except to just make this declaration.

And then the original method clone is protected. So, I can actually invoke it only from within the class only the class can clone itself, nothing else can clone it if you leave it that way. So, then you have to expand the scope to public in order to let other classes use this clone method.

And finally, there is this exception, which checks for this interface being implemented and that exception is thrown at the level of objects. So, as you override it, you have to continue throwing the same exception or you have to catch it. So, there is this further extra thing about this exception handling which comes in.

And it also means that typically when you invoke clone in your code, you cannot invoke it just as a function call, because it has the potential to throw this exception, so you must put it into a try block and do something with that. So, these are all these checks and balances that Java puts to make Cloneable somewhat problematic to use, so that the programmer using Cloneable uses it discretion.